

PlaceRecover: Layout-Invariant Point Cloud Recovery for Robust 3D Perception under LiDAR Placement Shifts

Benwu Wang[†], Zihang Wang[†], Wenkai Zhu, Haotong Luan, Dong Kong^{*}, Binghao Wang, Yiming Zhou, Kailin Lyu, Teng Yan, Haoyu Wang, Yaohua Liu, and Qiang Zhang

Abstract—3D perception is a core capability for autonomous vehicles and robotic systems, and its reliability under unseen conditions is crucial for deployment. As autonomous driving advances, multi-LiDAR configurations are increasingly adopted to improve perception coverage and redundancy. However, most LiDAR-based perception models are trained under a fixed, centrally mounted sensor layout, and their performance can degrade notably when deployed under heterogeneous LiDAR placements due to systematic shifts in viewpoint-induced sampling patterns. To address this, we propose PlaceRecover, a plug-and-play layout recovery network that transforms point clouds from diverse LiDAR layouts into a canonical Center layout, enabling direct reuse of Center-trained perception models without retraining. PlaceRecover employs an INR-PointTransformer encoder to jointly capture local geometry and global implicit layout representations, and a Layout-Aware PointTransformer decoder that explicitly injects layout context for robust recovery across sensor placements. Extensive experiments on Place3D demonstrate that PlaceRecover substantially improves recovery quality with CD reduced by 40.5% and F-score@0.2 increased by 24.4%, and consistently boosts downstream 3D object detection and semantic segmentation across multiple detectors and segmentors under layout shifts. These results indicate that PlaceRecover effectively mitigates perception degradation caused by LiDAR placement shifts and supports robust deployment for heterogeneous multi-LiDAR systems.

I. INTRODUCTION

Reliable 3D perception under complex real-world scenarios is a prerequisite for the safe deployment of autonomous driving and robotic systems [1], [2]. As autonomous driving

This work was supported in part by the Natural Science Foundation of Shandong Province under Grant ZR20240F106 and by the China Postdoctoral Science Foundation under Grant 2025M781569. ^{*}Corresponding author: Dong Kong.

[†]Benwu Wang and Zihang Wang contributed equally to this work.

Benwu Wang, Zihang Wang, and Wenkai Zhu are with the School of Instrument Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: wehang2023@163.com; wzhanggg@seu.edu.cn; 230248520@seu.edu.cn).

Haotong Luan is with Shenzhen Technology University, Shenzhen, China, and also with the Laboratory of Robotics, X-Institute, Shenzhen, China (e-mail: lht20060306@outlook.com).

Dong Kong is with the School of Electronic and Information Engineering, Tongji University, Shanghai 201804, China (e-mail: kongd.6696@163.com).

Binghao Wang and Qiang Zhang are with Jiangsu University of Science and Technology, China (e-mail: 18846969005@163.com; qzhang@just.edu.cn).

Yiming Zhou is with Laval University, Canada (e-mail: zhou.ym19@gmail.com; yiming.zhou.1@ulaval.ca).

Kailin Lyu is with the Institute of Automation, Chinese Academy of Sciences, China (e-mail: lvkailin2024@ia.ac.cn).

Teng Yan is with The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China (e-mail: YTupup@163.com).

Haoyu Wang is with Sichuan University, China (e-mail: haruka.haoyu@gmail.com).

Yaohua Liu is with the Guangdong Institute of Intelligence Science and Technology, China (e-mail: yaohua.liu@163.com).

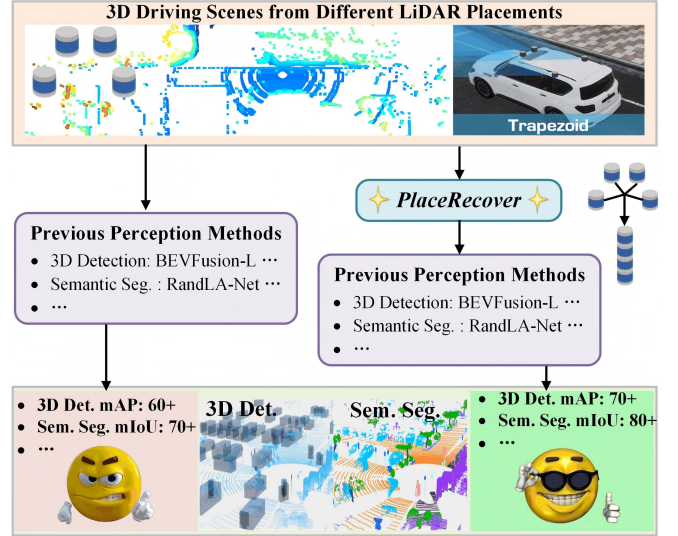


Fig. 1: PlaceRecover consistently improves point cloud perception under different LiDAR placements.

systems move toward large-scale production, multi-LiDAR configurations are widely adopted to improve perception coverage and system redundancy [3]. However, most publicly available LiDAR perception models and methods are designed and trained under centrally mounted sensor setups [18], [19], implicitly assuming fixed viewpoints and point cloud distributions. This assumption differs substantially from heterogeneous sensor layouts in real industrial deployments, making initial deployment, model transfer, and rapid adaptation on new platforms difficult, while also causing distribution shifts, performance degradation, and high data annotation and retraining costs.

While progress has been made in 3D object detection [6], [7] and semantic segmentation [8], [9], current methods still have limitations. Most state-of-the-art approaches assume fixed sensor layouts, leading to poor performance under non-ideal, especially unconventional, configurations. Existing models often struggle with adapting to different sensor configurations, leading to degraded point cloud quality and reduced performance. Additionally, spatial and contextual information, crucial for recovering point clouds from diverse layouts, is often neglected. Place3D [3] has shown that evaluating the impact of sensor placement on 3D scene understanding is a significant challenge, complicating the determination of optimal sensor configurations for better performance. However, to the best of our knowledge, prior work

has not addressed multi-sensor layout recovery for initial deployment, namely aligning point clouds from heterogeneous sensor layouts to a standard representation compatible with existing perception models without changing the backbone, thereby improving initial deployment performance.

To address these challenges, we propose PlaceRecover, a Transformer-based network for recovering point clouds from various LiDAR sensor layouts. PlaceRecover utilizes the INR-PointTransformer encoder for feature extraction, ensuring both geometric consistency and semantic coherence in point cloud recovery. The PointTransformer V3 (PTv3) [13] encoder extracts spatial features, while the network employs implicit neural representation (INR) [14] to learn implicit 3D representations, enhancing recovery quality and robustness. A layout-aware PointTransformer decoder, enhanced by the layout-context block, is used to recover point clouds from different sensor placements, considering global context and ensuring robust recovery across diverse environmental conditions and sensor configurations, as shown in Fig. 1.

In summary, our main contributions are as follows:

- We propose PlaceRecover, a point cloud recovery network for multi-LiDAR layout adaptation. As a plug-and-play front-end module, it recovers point clouds from heterogeneous layouts into a standard representation compatible with existing perception models, without modifying the perception backbone.
- We design an INR-PointTransformerV3 encoder and a layout-aware PointTransformer decoder to jointly model local geometric features, global implicit layout representations, and layout contextual information for robust point cloud layout recovery.
- We conduct extensive experiments on point cloud recovery, 3D object detection, and 3D semantic segmentation. Results show that PlaceRecover significantly improves initial deployment performance while maintaining efficient inference.

II. RELATED WORK

A. LiDAR Sensing and 3D Perception

LiDAR is a core sensing modality for 3D scene understanding in autonomous vehicles and mobile robots. With recent advances in deep learning, LiDAR-based perception has achieved strong performance in 3D object detection and semantic segmentation. For detection, representative methods include PointPillars [4], CenterPoint [6], and BEVFusion [7], which improve efficiency and accuracy via pillar-based encoding, center-based BEV detection, and multi-modal BEV fusion, respectively. For segmentation, RangeNet [16], RandLA-Net [8], and LaserMix [17] achieve strong performance through range-view projection, efficient point sampling and aggregation, and structured semi-supervised augmentation. LiDAR is also widely used in downstream tasks such as scene flow estimation [11] and place recognition [12]. However, most existing methods are developed under single-LiDAR, centrally mounted, or fixed sensor configuration assumptions, and thus do not explicitly address sensor layout variations,

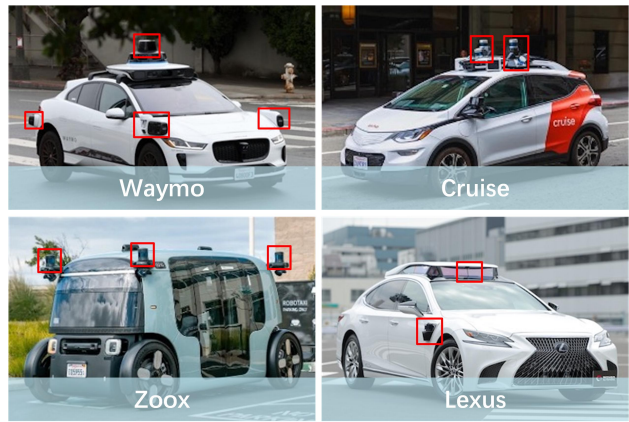


Fig. 2: Multi-LiDAR layout examples on different autonomous driving vehicles. Red boxes indicate the LiDAR mounting positions.

posing challenges for initial deployment and rapid adaptation on heterogeneous multi-LiDAR platforms.

B. Multi-Sensor Placement and Optimization

As autonomous driving systems move toward large-scale production, multi-LiDAR configurations are increasingly adopted to improve perception coverage and system redundancy. As shown in Fig. 2, different platforms employ substantially different LiDAR layouts: Waymo [10] uses multiple LiDARs for omnidirectional coverage; Cruise adopts a dual-LiDAR rooftop configuration for enhanced forward and long-range perception; Zoox adopts dual rooftop LiDARs together with dual tilted LiDARs to enhance surround coverage, side-view perception, and robustness in complex scenes; and Lexus combines a rooftop primary LiDAR with side-mounted LiDARs to improve lateral perception and near-field coverage. Place3D [3] further introduces LiDAR placement evaluation and optimization for 3D perception, showing that proper multi-LiDAR placement can improve 3D detection and semantic segmentation under diverse conditions. However, many advanced perception algorithms are trained on fixed-layout datasets such as KITTI [19] and nuScenes [18]. When directly deployed to platforms with different multi-LiDAR layouts, they often suffer from performance degradation due to viewpoint changes, point density shifts, and geometric observation offsets. This increases data collection, annotation, and retraining costs during initial deployment, and highlights the need for front-end adaptation and point cloud layout recovery under heterogeneous multi-sensor layouts.

III. PROPOSED METHOD

A. Overall Pipeline

Fig. 3 illustrates the overall pipeline of our PlaceRecover network. The network integrates the global geometric modeling capability of the PointTransformer V3, the implicit spatial relationship modeling features from INR, and layout contextual information, enabling robust point cloud recovery across various LiDAR placements.

Specifically, given a non-Center point cloud input $P_c \in \mathbb{R}^{N \times 4}$, where N is the number of points in the point cloud

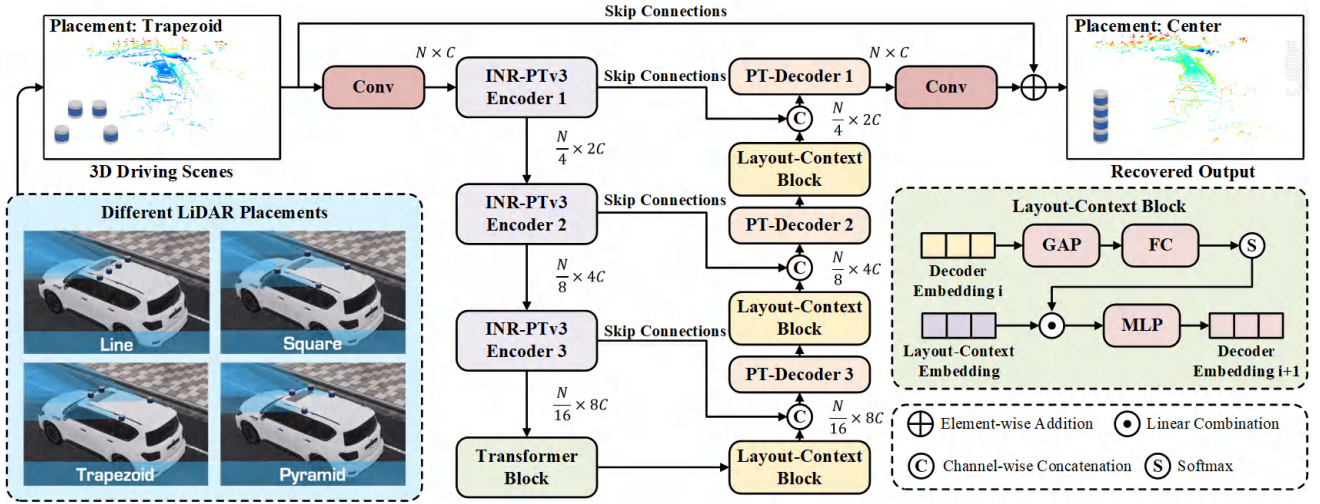


Fig. 3: Overall architecture of PlaceRecover, which employs INR-PointTransformer encoders, a Transformer block, and PointTransformer decoders with Layout-Context Blocks to recover point clouds across different LiDAR sensor placements.

and each point is represented by four features including its 3D coordinates (x, y, z) and intensity i , the point cloud is first passed through a convolutional block to extract features. This process produces a feature map $F_0 \in \mathbb{R}^{N \times C}$, where C denotes the number of feature channels. Subsequently, these features are refined through our carefully designed PlaceRecover restoration encoder-decoder architecture.

The encoder consists of three INR-PointTransformer V3 modules, each integrating self-attention mechanisms, spatial feature learning, and implicit neural representations. The PointTransformer V3 encoder first processes the input point cloud through multi-head self-attention and feed-forward layers, capturing complex spatial dependencies among points. In parallel, the INR component introduces learnable weight tokens that are concatenated with data tokens and jointly optimized within the Transformer. These weight tokens implicitly encode the mapping from different LiDAR sensor layouts to a canonical center layout, thereby enhancing structural consistency and geometric detail in the recovered features.

As the features pass through these layers, the spatial resolution progressively decreases, and the channel count increases, ultimately generating a compact latent representation $F_d \in \mathbb{R}^{N/16 \times 8C}$. An additional transformer module is applied on the bottleneck feature to capture spatial correlations, thereby enhancing the feature representation ability.

Then, a three-layer layout-aware (LA) PointTransformer decoder is applied to this compact latent representation, progressively recovering the spatial details of the point cloud. In each layer of the decoder, we introduce a layout-context block, which uses contextual information related to the sensor layout to refine the features. This block helps guide the recovery process, ensuring geometric consistency and robustness of point cloud restoration across various sensor configurations. Skip connections between the encoder and decoder preserve critical spatial details, ensuring high-fidelity recovery.

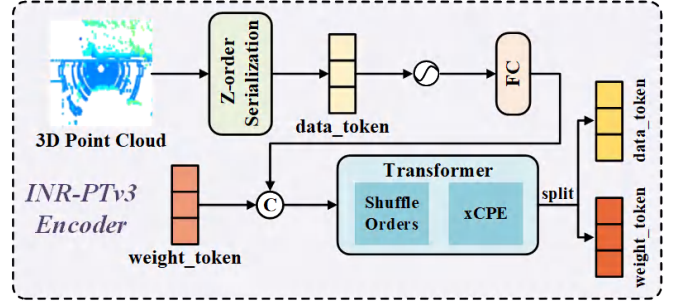


Fig. 4: Structure of the INR-PTv3 encoder, where serialized point-cloud data tokens are combined with weight tokens and refined by a Transformer with Shuffle Orders and xCPE.

B. INR-PointTransformer Encoder

Recovering point clouds under LiDAR placement shifts requires an encoder that captures local geometry while providing a compact global code to condition layout recovery. Hence, we propose an INR-PointTransformer encoder that combines PTv3-style serialized patch attention with an INR-Transformer weight-token mechanism, as shown in Fig. 4. Given a source-layout point cloud $P_{\text{src}} = \{(\mathbf{p}_n, i_n)\}_{n=1}^N$, a lightweight point-wise MLP stem first embeds each point into feature space, which is then fed into the following tokenization and Transformer blocks.

To organize the unordered points for Transformer modeling, we serialize them with a Z-order (Morton) space-filling curve in Fig. 5, avoiding explicit KNN graph construction while largely preserving spatial locality:

$$\text{Encode}(\mathbf{p}, b, g) = (b \ll K) \mid \text{Morton}(\lfloor \mathbf{p}/g \rfloor), \quad (1)$$

where b is the batch index, g is the quantization grid size, $\lfloor \mathbf{p}/g \rfloor \in \mathbb{Z}^3$ is the voxelized coordinate, $\text{Morton}(\cdot)$ returns the Z-order code, and K reserves sufficient bits to separate different batches. The serialized sequence is grouped into fixed-size patches. Each token is augmented with a learnable positional

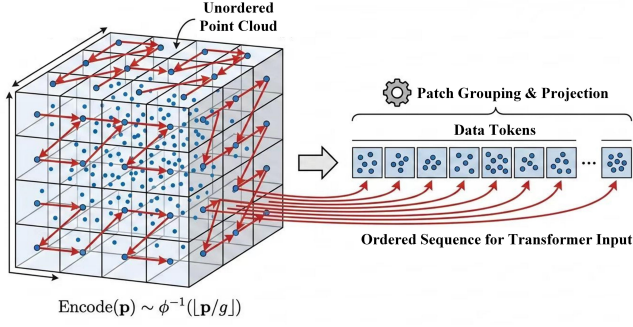


Fig. 5: Point cloud serialization with Z-order (Morton) codes, followed by patch grouping for Transformer tokens.

embedding and projected to a *data token*:

$$\text{data_token}_i = \text{FC}(\text{Proj}(\mathbf{p}_i) + e_i), \quad (2)$$

where $\text{Proj}(\cdot)$ maps coordinates to the token feature dimension and e_i is the learnable positional embedding. This serialization supports efficient patch attention with a larger receptive field.

In parallel, we introduce a small set of learnable *weight tokens* to parameterize a conditional implicit neural representation (INR) that captures a global, layout-adaptive mapping prior. Weight tokens are concatenated with data tokens and processed by the same Transformer, and outputs at the weight-token slots are mapped to INR weight columns:

$$\mathbf{W}_i = \text{FC}(\text{weight_token}_i), \quad (3)$$

with *weight grouping* to reduce computation:

$$\mathbf{w}_i = \text{normalize}(u_{\lfloor i/k \rfloor}) \cdot \bar{\mathbf{w}}_i, \quad (4)$$

where u_j is shared within each group of size k and $\bar{\mathbf{w}}_i$ is an independent scaling parameter. The resulting INR weights provide a compact global code that conditions the decoder via the context embedding CE : during recovery, the decoder upsamples to a set of query points and directly regresses per-point offsets conditioned on both local token features and CE .

Within each Transformer block, patch tokens undergo serialized patch attention, enhanced with xCPE implemented by sparse convolutions and residual connections. PTV3's Shuffle-Order interaction further enlarges the receptive field and reduces serialization bias, enabling effective integration of context. Finally, a bottleneck Transformer refines the compact latent representation, which is passed to the layout-aware decoder for point cloud recovery.

C. LA-PointTransformer Decoder

Point cloud reconstruction requires a decoder that not only recovers Center-aligned point clouds but also adapts to varying LiDAR layouts and environmental conditions, particularly with degraded data. We propose the LA-PointTransformer Decoder, which uses stepwise upsampling and feature refinement to enhance point cloud quality and adaptability to environmental and layout changes.

The decoder processes low-resolution point clouds with linear transformations, batch normalization (BN), and ReLU

activation, followed by feature interpolation to propagate low-resolution features to higher-resolution points. Skip connections fuse encoder features with high-resolution outputs, preserving key information. These features are then refined in the Point Transformer Block.

The PointTransformer block uses self-attention to capture spatial dependencies, with relative position encoding improving adaptability to LiDAR layouts. The core formula is:

$$y_i = \sum_{x_j \in X(i)} \rho \left(\gamma \left(\phi(x_i) - \psi(x_j) + \delta \right) \right) \odot (\alpha(x_j) + \delta) \quad (5)$$

Here, $X(i)$ is the set of k nearest neighbors of x_i , δ is the relative position encoding, γ is the attention function, and ρ is the softmax normalization.

To further enhance adaptability, the decoder introduces the layout-context Block, which dynamically adjusts features to handle degradation patterns. It extracts global information via global average pooling (GAP), computes a weight vector, and multiplies it with context embedding (CE) to generate adjusted features. Convolutional operations are then applied to refine spatial consistency.

The feature adjustment formula in the layout-context Block is:

$$\begin{aligned} \mathbf{W}_{wb} &= \text{Softmax}(\text{FC}(\text{GAP}(\mathbf{F}_{wb}))), \\ \mathbf{F}_{cb} &= \text{MLP}(\mathbf{W}_{wb} \cdot \mathbf{CE}). \end{aligned} \quad (6)$$

Here, $\mathbf{F}_{wb}$ represents features refined by the Point Transformer Block, $\mathbf{W}_{wb}$ is the GAP-extracted weight vector, CE is the context embedding, and $\mathbf{F}_{cb}$ is the adjusted feature incorporating degradation information.

The layout-context block significantly improves adaptability, enabling effective point cloud recovery under diverse environments and sensor configurations.

D. Loss Function

To train PlaceRecover for layout-to-Center recovery, we adopt a set-to-set objective to avoid imposing an arbitrary point ordering. Given the predicted point set $\hat{P} = \{(\hat{\mathbf{d}}_j, \hat{i}_j)\}_{j=1}^N$ and the Center reference $P = \{(\mathbf{d}_k, i_k)\}_{k=1}^N$, we use a Chamfer loss on coordinates and an intensity regression loss aligned by nearest neighbors:

$$\mathcal{L}_{CD}(\hat{P}, P) = \frac{1}{\hat{N}} \sum_j \min_k \|\hat{\mathbf{d}}_j - \mathbf{d}_k\|_2 + \frac{1}{N} \sum_k \min_j \|\mathbf{d}_k - \hat{\mathbf{d}}_j\|_2, \quad (7)$$

$$\mathcal{L}_{int}(\hat{P}, P) = \frac{1}{\hat{N}} \sum_j |\hat{i}_j - i_{\pi(j)}|, \quad \pi(j) = \arg \min_k \|\hat{\mathbf{d}}_j - \mathbf{d}_k\|_2. \quad (8)$$

The final training objective is $\mathcal{L} = \mathcal{L}_{CD} + \lambda \mathcal{L}_{int}$, where λ balances geometry and intensity, which encourages geometric alignment while preserving intensity consistency, and is robust to the unordered nature of point clouds.

IV. EXPERIMENTS

A. Datasets and Evaluation Protocol

Datasets. We validate the proposed PlaceRecover on the multi-layout LiDAR dataset Place3D [3]. This dataset provides synchronized point clouds acquired under four sensor

TABLE I: Recovery quality in the typical ROI region, reporting results before and after recovery and their absolute change, and the best and second-best values are highlighted in red and blue, respectively.

Shift	CD ↓				MHD ↓				F-score@0.2 ↑			
	Before	After	Change	Change (%)	Before	After	Change	Change (%)	Before	After	Change	Change (%)
Line → Center	0.2981	0.1736	-0.1245	-41.8	0.1380	0.1157	-0.0224	-16.2	0.629	0.795	0.166	26.4
Pyramid → Center	0.3037	0.1806	-0.1231	-40.5	0.1389	0.1176	-0.0213	-15.3	0.633	0.777	0.143	22.6
Trapezoid → Center	0.2912	0.1770	-0.1142	-39.2	0.1417	0.1187	-0.0230	-16.2	0.628	0.781	0.153	24.3
Mean	0.2977	0.1771	-0.1206	-40.5	0.1395	0.1173	-0.0222	-15.9	0.630	0.784	0.154	24.4

placements: a Center configuration and three non-Center configurations (e.g., Line, Pyramid and Trapezoid) of each frame. In addition, it provides 3D object detection annotations for three classes and semantic segmentation labels for 21 classes, enabling evaluation on various perception tasks. Because all four layouts are captured simultaneously within the same scene, the Center-layout point cloud serves as ground-truth (GT) supervision for point cloud recovery.

Layouts Recovery. The recovery quality of non-Center layouts is quantified by Chamfer Distance (CD) with Center layout [27]. To further reflect robustness, we also report Modified Hausdorff Distance (MHD), defined as the median nearest-neighbor distance between two point sets, and F-score@ τ , i.e., the fraction of points within a tolerance threshold τ [28].

Downstream Perception. To isolate the contribution of the layout shift recovery, downstream models are trained exclusively on the Center layout, and evaluated on non-Center layouts without any adaptation. We consider two evaluation streams for each non-Center layout: a raw stream that directly applies the model to the non-Center input, and a recovered stream that first applies PlaceRecover and then feeds recovered data into the same model. For 3D object detection [3], we report Average Precision (AP) in both BEV and full 3D at IoU thresholds of 0.5, with class-wise AP for the three categories and mean AP (mAP). For semantic segmentation, we report mean IoU (mIoU) over 21 classes, along with mean accuracy (mAcc) and expected calibration error (ECE) to assess prediction correctness and confidence calibration. All downstream results are computed with the same evaluation protocol across layouts and settings to ensure fair comparison.

B. Implementation Details

Our PlaceRecover is trained in a supervised manner using grouped non-Center/Center point clouds, which is implemented in PyTorch on top of MMDetection3D [25]. The training is conducted on a single NVIDIA RTX 4090 GPU for 10 epochs with a batch size of 1, using Adam with an initial learning rate of 5×10^{-5} and a cosine annealing schedule. The encoder is derived from the simplified PointTransformerV3 [13]. For recovery evaluation, we report the F-score at $\tau = 0.2$ m, computed in a typical ROI defined by range ≥ 10 m with $x, y \in [-100, 100]$ m and $z \in [-3, 1]$ m. We also report the improvement Δ to quantify the recovery effect.

We evaluated multiple 3D detectors and segmenters to demonstrate that PlaceRecover is generalized across architectures. In detection pipeline, two widely used PointPillars [4], CenterPoint [6] and two strong modern detectors BEVFusion

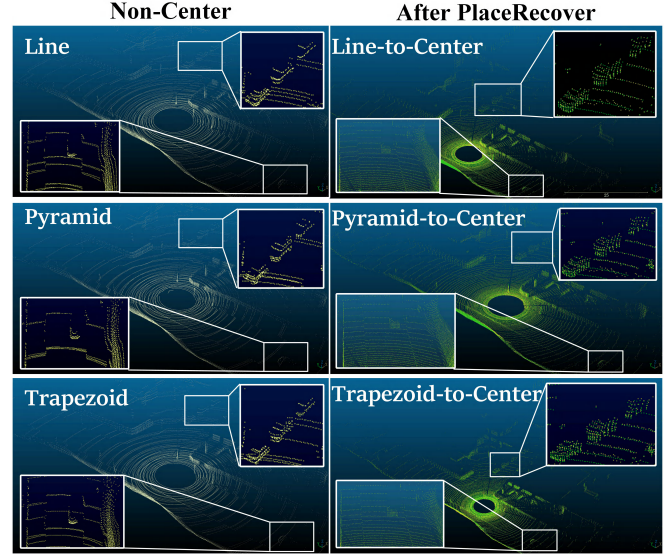


Fig. 6: Visualization of point cloud recovery from various layout.

[7], FSTR [29] are included. All detectors are trained only on Center layout using an identical schedule with 20 epochs, batch size 4, initial learning rate 0.002 with the AdamW optimizer, a cosine annealing learning-rate schedule, weight decay 0.01. Following Place3D [3], we train the semantic segmentation model including MinkUNet [30], SPVCNN [31], PolarNet [32], and Cylinder3D [33] exclusively on Center-layout data, and evaluate by transferring predicted labels via 3D nearest-neighbor association to the Center reference against the Center GT labels. At test time, model weights are kept fixed, only the input is replaced by the recovered output in the PlaceRecover setting, ensuring a fair comparison.

C. Main Results

1) *Point Cloud Recovery:* We evaluate the point cloud recovery quality of PlaceRecover, and summarize the quantitative results in Table I. Across all three placement-to-Center shifts, PlaceRecover consistently decreases CD and MHD while increasing F-score. On average, CD is reduced from 0.2977 to 0.1771 (-40.5%), MHD drops from 0.1395 to 0.1173 (-15.9%), and F-score improves from 0.630 to 0.784 (+24.4%). The reduction in CD indicates improved global geometric agreement with the Center reference, while the decrease in MHD suggests that local worst-case deviations are effectively suppressed. Meanwhile, the gain in F-score implies a substantially higher fraction of points can be matched within

TABLE II: Center-only training results for detection and semantic segmentation across different LiDAR layouts, with and without PlaceRecover (+PR). The best and second-best values are highlighted in red and blue, respectively.

Layout	Detector	BEV AP@0.5 $\uparrow$				3D AP@0.5 $\uparrow$				Segmentor	mIoU $\uparrow$	mAcc $\uparrow$	ECE $\downarrow$
		Car	Ped	Bic	mAP	Car	Ped	Bic	mAP				
Center	PointPillars	0.589	0.303	0.341	0.411	0.465	0.194	0.271	0.310	MinkUNet	0.657	0.724	0.041
	CenterPoint	0.671	0.354	0.418	0.481	0.558	0.287	0.288	0.378	SPVCNN	0.671	0.744	0.034
	BEVFusion-L	0.628	0.433	0.372	0.478	0.525	0.319	0.322	0.389	PolarNet	0.710	0.760	0.033
	FSTR	0.642	0.389	0.457	0.496	0.553	0.277	0.293	0.374	Cylinder3D	0.727	0.792	0.041
Line	PointPillars	0.536	0.223	0.273	0.348	0.400	0.105	0.190	0.232	MinkUNet	0.598	0.676	0.063
	CenterPoint	0.615	0.283	0.337	0.415	0.492	0.212	0.200	0.298	SPVCNN	0.602	0.690	0.043
	BEVFusion-L	0.574	0.355	0.305	0.409	0.460	0.229	0.249	0.318	PolarNet	0.643	0.704	0.047
	FSTR	0.576	0.317	0.385	0.421	0.482	0.197	0.215	0.296	Cylinder3D	0.661	0.740	0.057
Line+PR	PointPillars	0.579	0.296	0.334	0.402	0.457	0.172	0.261	0.291	MinkUNet	0.650	0.718	0.048
	CenterPoint	0.664	0.346	0.408	0.472	0.551	0.276	0.278	0.368	SPVCNN	0.657	0.736	0.036
	BEVFusion-L	0.618	0.425	0.363	0.467	0.516	0.299	0.311	0.380	PolarNet	0.700	0.754	0.041
	FSTR	0.633	0.379	0.451	0.487	0.546	0.267	0.283	0.364	Cylinder3D	0.718	0.783	0.047
Trapezoid	PointPillars	0.518	0.213	0.259	0.335	0.380	0.097	0.173	0.212	MinkUNet	0.588	0.669	0.062
	CenterPoint	0.609	0.271	0.338	0.405	0.484	0.192	0.203	0.291	SPVCNN	0.593	0.683	0.049
	BEVFusion-L	0.558	0.342	0.288	0.395	0.450	0.219	0.229	0.296	PolarNet	0.634	0.690	0.052
	FSTR	0.576	0.295	0.363	0.409	0.476	0.167	0.195	0.277	Cylinder3D	0.650	0.732	0.055
Trapezoid+PR	PointPillars	0.574	0.286	0.330	0.393	0.427	0.166	0.245	0.278	MinkUNet	0.648	0.718	0.047
	CenterPoint	0.662	0.345	0.408	0.472	0.547	0.271	0.279	0.360	SPVCNN	0.665	0.734	0.042
	BEVFusion-L	0.612	0.420	0.364	0.460	0.500	0.295	0.311	0.369	PolarNet	0.701	0.751	0.040
	FSTR	0.632	0.378	0.438	0.481	0.529	0.241	0.271	0.349	Cylinder3D	0.720	0.785	0.048
Pyramid	PointPillars	0.504	0.205	0.251	0.319	0.370	0.085	0.175	0.207	MinkUNet	0.581	0.644	0.059
	CenterPoint	0.593	0.249	0.315	0.380	0.474	0.168	0.170	0.272	SPVCNN	0.587	0.681	0.048
	BEVFusion-L	0.548	0.341	0.269	0.385	0.433	0.222	0.208	0.290	PolarNet	0.630	0.694	0.057
	FSTR	0.560	0.293	0.360	0.406	0.466	0.170	0.183	0.269	Cylinder3D	0.658	0.729	0.060
Pyramid+PR	PointPillars	0.561	0.278	0.335	0.386	0.429	0.158	0.261	0.278	MinkUNet	0.638	0.711	0.050
	CenterPoint	0.649	0.336	0.388	0.455	0.530	0.254	0.234	0.337	SPVCNN	0.651	0.734	0.040
	BEVFusion-L	0.616	0.415	0.342	0.454	0.498	0.296	0.272	0.359	PolarNet	0.702	0.750	0.043
	FSTR	0.632	0.379	0.447	0.482	0.535	0.252	0.269	0.356	Cylinder3D	0.721	0.785	0.050

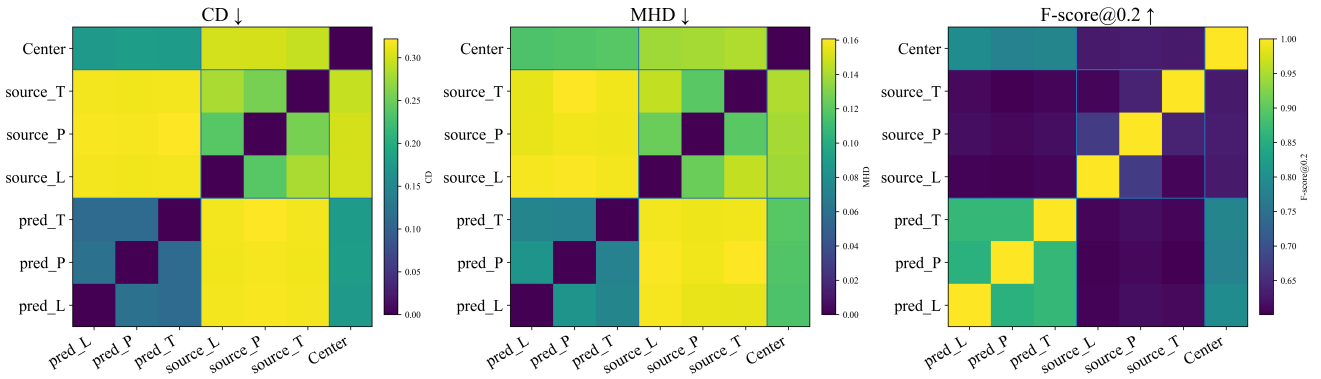


Fig. 7: Pairwise metric heatmaps among raw non-Center point clouds (source), recovered point clouds (pred), and the Center reference in the typical ROI region.

the $\tau=0.2$ m tolerance, reflecting enhanced fine-grained local correspondence. The improvement trends are consistent across three placements, indicating that PlaceRecover generalizes reliably under placement variations.

Furthermore, we diagnose the cross-placement distribution shift and to verify whether PlaceRecover consistently reduces this gap via recovery as shown in Fig 7. With the block-wise layout, the source-to-Center entries show larger CD/MHD

and lower F-score@0.2, confirming a pronounced mismatch induced by non-Center placements. In contrast, the pred-to-Center entries exhibit substantially reduced CD/MHD and increased F-score@0.2, indicating that recovered point clouds are brought much closer to the Center reference. Moreover, the pred-pred block is more compact than the source-source block, suggesting improved cross-placement consistency toward a shared Center-aligned representation. Taken together with

TABLE III: Ablation of PlaceRecover on the Line layout. We report point-cloud recovery quality and downstream detection accuracy using PointPillars. Baseline: raw Line inputs without recovery, Full: Line+PlaceRecover.

Variant	CD ↓	MHD ↓	F-score@0.2 ↑	BEV AP@0.5 ↑				3D AP@0.5 ↑			
				Car	Ped	Bic	mAP	Car	Ped	Bic	mAP
Baseline (w/o recovery)	0.2981	0.1380	0.629	0.536	0.223	0.273	0.348	0.400	0.105	0.190	0.232
PlaceRecover (Full)	0.1736	0.1157	0.795	0.579	0.296	0.334	0.402	0.457	0.172	0.261	0.291
w/o INR module	0.2057	0.1216	0.744	0.566	0.284	0.322	0.391	0.446	0.160	0.248	0.285
w/o Layout-Context block	0.2173	0.1239	0.731	0.559	0.279	0.318	0.385	0.440	0.156	0.244	0.280
w/o Point-wise Encoder	0.2110	0.1227	0.737	0.563	0.282	0.320	0.388	0.443	0.158	0.246	0.282

Table I, Fig 7 provides coherent evidence that PlaceRecover improves both global alignment and local point-level matching quality, offering a stronger geometric basis for downstream models that trained on the popular Center distribution.

2) *Downstream Perception Results:* Table II shows that training on the Center layout only leads to clear performance drops when testing on shifted layouts such as Line, Trapezoid, and Pyramid. After applying PlaceRecover, both detection and segmentation improve consistently across all model combinations, indicating that the recovered point clouds better match the Center-trained downstream models. For detection, the gains are stable but moderate. In most cases, BEV and 3D mAP increase by roughly 5 points, with larger improvements observed under stronger layout shift, especially for Trapezoid and Pyramid. For semantic segmentation, the gains are similarly pronounced, particularly on Trapezoid and Pyramid, suggesting that recovery is effective at restoring far-range structures and improving semantic consistency. Overall, PlaceRecover mitigates layout-shift degradation without changing downstream training, while the results remain below the Center upper bound as expected due to irrecoverable layout-dependent missing geometry.

3) *Qualitative Visualization:* We provide qualitative visualizations to complement the quantitative results by comparing raw non-Center point clouds and PlaceRecover outputs overlaid with the Center ground truth. As illustrated in Fig. 6 and Fig. 7, PlaceRecover improves geometric consistency with the Center reference: global misalignment and placement-induced local offsets are reduced, object boundaries become more continuous, and far-range sparse regions exhibit fewer holes and artifacts. We further evaluate downstream transfer by feeding recovered point clouds into Center-trained detectors and segmentors. As shown in Fig. 8 and Fig. 9, using raw non-Center inputs leads to missed detections, biased box localization, and fragmented segmentation, especially in sparse or boundary regions. Replacing the input with recovered point clouds alleviates these issues, yielding higher recall, better box-to-geometry alignment, and more coherent segmentation. Failure cases still occur under extremely sparse, occluded, or thin-structure scenarios, leaving room for improvement.

D. Ablation Study

Table III shows that PlaceRecover consistently improves both recovery quality and downstream detection over the no-recovery baseline. It achieves lower CD/MHD and a higher

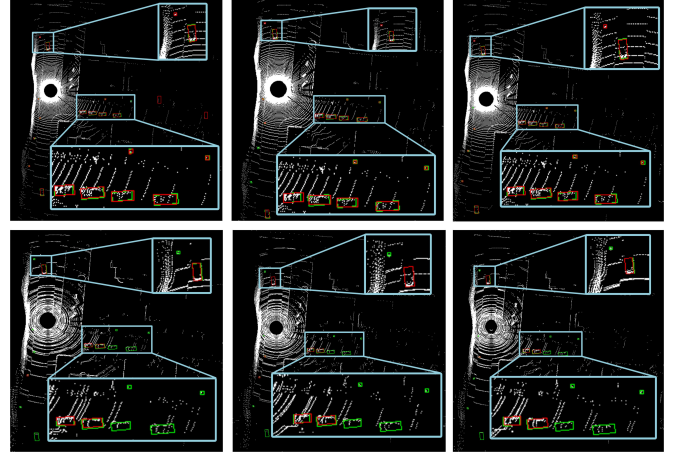


Fig. 8: Qualitative comparison of downstream 3D detection under layout shift. (Top: Recover, Bottom: Source).

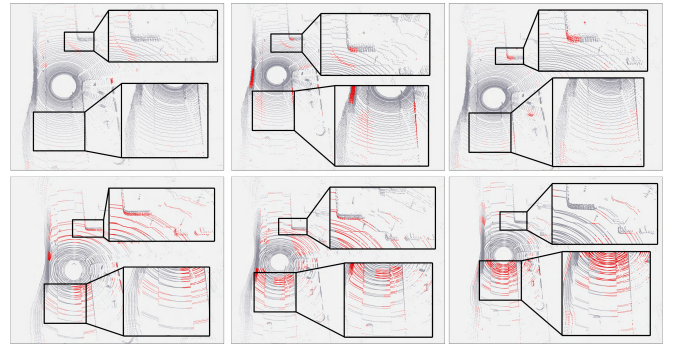


Fig. 9: Qualitative error map comparison of downstream semantic segmentation under layout shift. Red denotes misclassified points.

far-range F-score@0.2, and these gains transfer to detection, with BEV mAP increasing from 0.348 to 0.402 and 3D mAP from 0.232 to 0.291. The ablations further attribute the improvements to key design choices: removing the INR module or the Layout-Context block degrades CD/MHD, suppresses F-score@0.2, and reduces mAP, indicating that implicit refinement and layout-conditioned alignment are both critical for correcting placement-induced shifts. Replacing the Point-wise Encoder with a range-based representation also lowers recovery and detection performance, suggesting that point-level encoding is important for preserving fine geometric details that downstream detectors rely on.

E. Runtime Analysis

As a plug-and-play pre-processing module, PlaceRecover can be inserted before existing detectors or segmentors without changing their architectures. On a desktop-class GPU, it adds about 55 ms inference time per frame and contains roughly 38M parameters. The overhead mainly comes from the encoder-bottleneck-decoder pipeline and attention-based feature modeling, which are essential for capturing long-range context and producing layout-consistent geometry. In ongoing work, we are investigating lightweight variants through sparsification and further evaluating robustness under different point densities and sensor conditions to improve the efficiency-adaptability trade-off.

V. CONCLUSION

This paper presents PlaceRecover, a Transformer-based point cloud recovery network for robust LiDAR sensor layout adaptation, designed to address the challenge of initial deployment in multi-LiDAR perception systems. By integrating an INR-PointTransformerV3 encoder and an LA-PointTransformer decoder, PlaceRecover models local geometry, global implicit layout representations, and layout context to enable robust point cloud recovery across sensor layouts and improve downstream 3D object detection and semantic segmentation. Experiments show that PlaceRecover significantly improves initial deployment performance while maintaining efficient inference, demonstrating its effectiveness for fast deployment in multi-LiDAR systems. We hope it can serve as a general, low-cost front-end adaptation solution for heterogeneous sensor configurations and inspire further research on robust multi-LiDAR perception.

REFERENCES

- [1] Z. Song, L. Liu, *et al.*, “Robustness-aware 3D object detection in autonomous driving: A review and outlook,” *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 15 407–15 436, 2024.
- [2] Y. Chae, H. Kim, *et al.*, “Towards robust 3D object detection with LiDAR and 4D radar fusion in various weather conditions,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 162–15 172.
- [3] Y. Li, L. Kong, *et al.*, “Is your LiDAR placement optimized for 3D scene understanding?” *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 34 980–35 017, 2024.
- [4] A. H. Lang, S. Vora, *et al.*, “PointPillars: Fast encoders for object detection from point clouds,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 12 697–12 705.
- [5] X. Chen, T. Zhang, *et al.*, “FUTR3D: A unified sensor fusion framework for 3D detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 172–181.
- [6] T. Yin, X. Zhou, *et al.*, “Center-based 3D object detection and tracking,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 11 784–11 793.
- [7] T. Liang, H. Xie, *et al.*, “BEVFusion: A simple and robust LiDAR-camera fusion framework,” *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 10 421–10 434, 2022.
- [8] Q. Hu, B. Yang, *et al.*, “RandLA-Net: Efficient semantic segmentation of large-scale point clouds,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 108–11 117.
- [9] T. Cortinhal, G. Tzelepis, and E. E. Aksoy, “SalsaNext: Fast, uncertainty-aware semantic segmentation of LiDAR point clouds for autonomous driving,” in *Proc. Int. Symp. Vis. Comput. (ISVC)*, Cham: Springer, 2020, pp. 207–222.
- [10] P. Sun, H. Kretschmar, *et al.*, “Scalability in perception for autonomous driving: Waymo Open Dataset,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 2446–2454.
- [11] Y. Lin and H. Caesar, “ICP-Flow: LiDAR scene flow estimation with ICP,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 501–15 511.
- [12] D. Kong, X. Li, *et al.*, “SC-LPR: Semantically consistent LiDAR place recognition based on chained cascade network in long-term dynamic environments,” *IEEE Trans. Image Process.*, vol. 33, pp. 2145–2157, 2024.
- [13] X. Wu, L. Jiang, *et al.*, “Point Transformer V3: Simpler faster stronger,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 4840–4851.
- [14] Y. Chen and X. Wang, “Transformers as meta-learners for implicit neural representations,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Cham: Springer, 2022, pp. 170–187.
- [15] J. S. K. Hu, T. Kuai, *et al.*, “Point density-aware voxels for LiDAR 3D object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 8469–8478.
- [16] A. Milioto, I. Vizzo, *et al.*, “RangeNet++: Fast and accurate LiDAR semantic segmentation,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, IEEE, 2019, pp. 4213–4220.
- [17] L. Kong, J. Ren, *et al.*, “LaserMix for semi-supervised LiDAR semantic segmentation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 21 705–21 715.
- [18] H. Caesar, V. Bankiti, *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 11 621–11 631.
- [19] A. Geiger, P. Lenz, *et al.*, “Vision meets robotics: The KITTI dataset,” *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [20] A. W. Reed, H. Kim, *et al.*, “Dynamic CT reconstruction from limited views with implicit neural representations and parametric motion fields,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 2258–2268.
- [21] M. Hui, Z. Wei, *et al.*, “MicroDiffusion: Implicit representation-guided diffusion for 3D reconstruction from limited 2D microscopy projections,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 11 460–11 469.
- [22] K. Haydarov, A. Muhamed, *et al.*, “Adversarial text to continuous image generation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 6316–6326.
- [23] I. Skorokhodov, S. Ignatyev, *et al.*, “Adversarial generation of continuous images,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 10 753–10 764.
- [24] RoboSense Challenge 2025 Organizers, “RoboSense Challenge 2025: Track 3 – Sensor Placement,” 2025. [Online]. Available: <https://robosense2025.github.io/track3>
- [25] MMDetection3D Contributors, “MMDetection3D: OpenMMLab next-generation platform for general 3D object detection,” <https://github.com/open-mmlab/mmdetection3d>, 2020.
- [26] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [27] H. Fan, H. Su, *et al.*, “A point set generation network for 3D object reconstruction from a single image,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 605–613.
- [28] Z. Huang, Y. Wen, *et al.*, “Surface reconstruction from point clouds: A survey and a benchmark,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 9727–9748, 2024.
- [29] D. Zhang, Z. Zheng, *et al.*, “Fully sparse transformer 3-D detector for LiDAR point cloud,” *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–12, 2023.
- [30] C. Choy, J. Gwak, *et al.*, “4D spatio-temporal ConvNets: Minkowski convolutional neural networks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, June 2019, pp. 3075–3084.
- [31] H. Tang, Z. Liu, *et al.*, “Searching efficient 3D architectures with sparse point-voxel convolution,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, ser. LNCS, vol. 12373, Springer, 2020, pp. 685–702.
- [32] Y. Zhang, Z. Zhou, *et al.*, “PolarNet: An improved grid representation for online LiDAR point clouds semantic segmentation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, June 2020, pp. 9601–9610.
- [33] X. Zhu, H. Zhou, *et al.*, “Cylindrical and asymmetrical 3D convolution networks for LiDAR segmentation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, June 2021, pp. 9939–9948.